Phase 4 limits · page 1 – my 3 limit tests on app01

same pattern every time: sudo systemd-run --scope -p <LIMIT> stress-ng <LOAD> → watch what the kernel does

make a BOX	→	put a LIMIT on it	→	run fake LOAD inside	→	read the PROOF
`systemd-run --scope`		`-p MemoryMax / MemoryHigh / CPUQuota`		`stress-ng --vm / --cpu`		`journalctl · top · cpu.max`

TEST 1 – hard memory limit

limit: **`MemoryMax=100M MemorySwapMax=0`**

load: `stress-ng --vm 1 --vm-bytes 300M`

(fake program to test the limit)

What happened

- wants ~340 MB
- reaches ~96 MB = the wall
- no swap allowed → nowhere to go

KILLED

by the OOM killer (7× – new PID each time)

Proof I saw

```
journalctl -k … | grep -i -E "oom|killed"
"Killed process … CONSTRAINT_MEMCG"
parent survived: "successful run"
```

like a full glass: no more fits → thrown out

remember: **MemoryMax = KILL**

memory has no "wait" – it must free space

TEST 2 – soft memory limit

limit: **`MemoryHigh=100M (swap allowed)`**

load: `stress-ng --vm 1 --vm-bytes 200M`

(fake program to test the limit)

What happened

- wants 200 MB
- above 100 MB: kernel pushes
- the extra into SWAP on disk

SLOWED

not killed – finished in 22.17 s, not 20 s

Proof I saw

```
journalctl -k … | grep -i killed
→ prints NOTHING
"successful run completed in 22.17s"
```

like a full desk: extra papers go to the shelf

remember: **MemoryHigh = SLOW**

kernel moves memory to disk instead

TEST 3 – CPU limit

limit: **`CPUQuota=20%`**

load: `stress-ng --cpu 1`

(fake program to test the limit)

What happened

- no limit: 100 % (one full core)
- with limit: may run 20 ms
- in every 100 ms, then WAITS

THROTTLED

top: 100 % → 20.5 % · never killed

Proof I saw

```
top → stress-ng %CPU 20.5
cpu.max → "20000 100000"
2 boxes → each got its own 20 %
```

like mobile data after the limit: slower, not cut

remember: **CPUQuota = WAIT**

CPU time can always be given later

Page 2 – when the hard wall is hit: who is killed, and how to read the proof

test 1 continued · the OOM score · protecting services · what systemd does after · console tricks

A · 6 services share one box (MemoryMax=1G) – total 1070 MB → over

web uses 120 MB · adj 0 **survives**	**db** uses 380 MB · adj 0 **KILLED – biggest user**	**cache** uses 150 MB · adj 0 **survives**
sshd uses 10 MB · adj -1000 **protected – never**	**worker** uses 90 MB · adj 0 **survives**	**6th: report** uses 300 MB · adj 0 **survives**

the 6th service pushed the box over – but "db" dies: the kernel kills the HIGHEST SCORE, mostly the biggest memory user – predictable, not luck, not "the newest one"

B · Who does what

1	**KERNEL**	sees the box hit MemoryMax
2	**KERNEL**	scores every process in THIS box
3	**KERNEL**	kills the highest score
4	**systemd**	marks: failed (Result: oom-kill)
5	**systemd**	restarts ONLY if Restart= is set
6	**systemd**	OOMPolicy=stop → ends whole service
		OOMPolicy=continue → rest keeps running

systemd does NOT choose – the kernel does

C · The OOM score – final score = memory part (kernel measures) + oom_score_adj (I set: -1000 … +1000)



checked on app01:

`cat /proc/$$/oom_score`	**667**
`cat /proc/$$/oom_score_adj`	**0**
`pgrep -o sshd`	**966**
`…/$(pgrep -o sshd)/oom_score_adj`	**-1000**

protect an important service: `OOMScoreAdjust=-1000 (or -500) in its unit file · live test: echo -500 | sudo tee /proc/<PID>/oom_score_adj`

make one die first: `OOMScoreAdjust=+1000 (stress-ng sets this on itself – that is why it was picked)`

careful · best design: **protect only the few that matter (if all are protected nothing can be freed) · best: one box per service**

D · Reading the proof – 3 commands joined by pipes (the output of one goes into the next)

```
sudo journalctl -k --since "5 min ago" | grep -i -E "oom|killed" | tail -5
```

`journalctl -k --since "5 min ago"` the kernel's messages, last 5 min	→	**`grep -i -E "oom|killed"`** only lines with oom OR killed (any case)

→ **`tail -5`**
only the newest 5 lines

E · Console tricks

`cmd & sleep 2; top`	& = background · sleep 2 = let it start · ; = then · top = watch it live
`sudo -v`	type the password first, so a background sudo command is not stuck waiting
`…/system.slice/run-*.scope/cpu.max`	a box folder exists only WHILE its test runs · slash, not dash · Tab completes paths
`Alt+F1 / Alt+F2 · sudo chvt 2`	switch between console 1 and 2 (click inside the VM first)

Final paragraph – Phase 4 limits, the whole evening: one pattern runs every test – systemd-run --scope makes a temporary box, -p puts a limit on it, stress-ng loads it, and I read the proof. MemoryMax is a hard wall: the job is KILLED (test 1, ~96 MB, CONSTRAINT_MEMCG). MemoryHigh is a soft limit: the job is SLOWED into swap (test 2, 22.17 s, no kill). CPUQuota makes the box WAIT – 20 ms of every 100 ms (test 3, 100 % → 20.5 %, cpu.max 20000 100000); each box has its own limit. When a shared box hits its wall, the KERNEL scores every process in that box – memory used plus oom_score_adj – and kills the highest, usually the biggest user, not the newest. On app01 my bash scores 667 (normal) and sshd is protected at -1000, so I can always log in. systemd only reacts: it marks the service failed (oom-kill), restarts it only if Restart= is set, and OOMPolicy decides whether the rest of that service stops or continues. I can protect a service with OOMScoreAdjust=-1000 in its unit file; the best design is one box per service. MemoryMax = kill · MemoryHigh = slow · CPUQuota = wait.